

TECHNICAL DESIGN DOCUMENT

Securing Service-to-Service Communication with Mutual TLS

Certificate Authorities · Trust Stores · Certificate Chains · Hostname Validation · Hardened TLS Policy · Automated Certificate Rotation

Every component and every workflow explained in plain English, with full architecture diagrams

Field	Detail
Document type	Architecture and operations reference
Subject	Mutual TLS (mTLS) for internal service-to-service traffic
Written for	Engineers, reviewers, auditors and managers — no prior cryptography knowledge assumed
Covers	PKI design, trust distribution, certificate validation, TLS hardening, secret management, automated rotation, monitoring and incident response
Diagrams	Figure 1 — Overall architecture; Figure 2 — Handshake sequence; Figure 3 — Rotation lifecycle

Contents

- 1 The Project Explained Simply** — the problem, the fix, and an everyday analogy
- 2 Plain-English Vocabulary** — every term used in this document, defined once
- 3 Architecture Diagram, Explained Piece by Piece** — Figure 1 and a walkthrough of every box and arrow
- 4 Every Component in Detail** — eighteen components, end to end
- 5 Workflow A — The mTLS Handshake** — Figure 2 and what happens on every single connection
- 6 Workflow B — Automated Certificate Rotation** — Figure 3, the renewal loop, and CA rollover
- 7 The Remaining Workflows** — building the PKI, onboarding, rollout, revocation
- 8 Failure Modes and Troubleshooting** — what breaks, what the error means, how to fix it
- 9 Design Decisions and Why They Were Made**
- 10 Operational Checklist**
- 11 Glossary**

1. The Project Explained Simply

1.1 The problem this solves

Inside a company, software is not one big program. It is dozens or hundreds of small programs, called services, that constantly call one another over the network. An orders service calls a payments service. A payments service calls a fraud service. This chatter is where the most sensitive data in the business actually lives while it is moving.

Historically that internal traffic was treated as safe simply because it was inside the company network. The reasoning was: the firewall keeps attackers out, so anything already inside must be friendly. That assumption breaks in three ways.

- **Nobody checks who is calling.** If a service accepts any request that arrives on its port, then any process that reaches that port — a compromised container, a misconfigured test job, an attacker who got a foothold anywhere — can call it and be served.
- **Traffic can be read in transit.** Plain HTTP inside the network is readable by anything sitting on the path: a logging agent, a network tap, a cloud misconfiguration, a compromised node.
- **Nobody checks who is answering.** A caller that simply trusts a hostname can be redirected to an impostor by DNS poisoning or a bad routing rule, and will happily hand its data to the wrong machine.

1.2 The solution in one paragraph

This project makes every internal connection mutually authenticated and encrypted using mutual TLS, usually shortened to mTLS. Each service is given its own cryptographic identity in the form of a certificate. When two services talk, both of them prove who they are before a single byte of application data is exchanged, and everything after that is encrypted. Because certificates expire on purpose, the whole issue-renew-deliver-reload cycle is fully automated through a secret management system, so that certificates are replaced quietly in the background long before they can expire and cause an outage.

The one-sentence version

Every service is given a tamper-proof ID card signed by a trusted authority; both sides check each other's card before talking; the conversation is encrypted; and the ID cards renew themselves automatically before they run out.

1.3 The everyday analogy

Imagine a secure government building where two staff members need to exchange confidential files.

- **The passport office is the Certificate Authority (CA).** It is the one body everyone agrees is allowed to issue valid ID. It keeps its master stamp locked in a safe.
- **The passport is the certificate.** It states a name, has a photo, carries an expiry date, and is stamped by the passport office so it cannot be forged.

- **The signature you can only produce yourself is the private key.** Anyone can look at your passport, but only you can sign in your own handwriting. Showing the passport is not enough; you must also sign in front of the other person to prove it is really yours and not stolen.
- **The list of passport offices the guard recognises is the truststore.** The guard does not memorise every citizen. He only memorises which passport offices are legitimate, and then trusts any passport those offices stamped.
- **Both people showing ID is what makes it mutual.** In ordinary web browsing only the website proves its identity to you. Here, both parties must show ID to each other. That is the whole difference between TLS and mutual TLS.
- **The sealed briefcase is encryption.** Once identities are confirmed, the files travel in a locked case that only those two can open, so nobody in the corridor can read or swap the contents.
- **Passport renewal is certificate rotation.** Passports expire. A person who forgets to renew is turned away at the door. So instead of relying on memory, the building runs an automatic renewal service that issues a fresh passport at two-thirds of the old one's life, hands it over, and lets the old one stay valid for a while so nobody is ever caught without one.

1.4 What was actually built

1. **A certificate authority hierarchy** — an offline root CA that is kept locked away, and an online intermediate CA that does the day-to-day signing. This means the signing machine can be replaced after a compromise without having to reconfigure every service in the company.
2. **A trust store distributed to every service** — a single, version-controlled bundle file listing exactly which certificate authorities are acceptable. Anything signed by anyone else is rejected.
3. **Correct chain and hostname validation** — each side rebuilds the chain from the certificate it was shown back to a CA it trusts, checks the dates, checks that the name in the certificate is the service it actually intended to reach, and checks the certificate has not been revoked.
4. **A hardened TLS policy** — modern protocol versions only, strong authenticated ciphers only, no legacy fallbacks, and identical settings on both the client and the server side so the two cannot silently negotiate down to something weak.
5. **Fully automated certificate rotation** — a controller watches every certificate, renews it at roughly two-thirds of its lifetime, stores it in the secret manager, delivers it to the workload, and triggers an in-place reload with no restart and no dropped connections.
6. **Monitoring that makes expiry impossible to miss** — days-until-expiry is exported as a metric for every certificate in the estate, with alerts that fire weeks before anything can break.

1.5 What the business gets out of it

Outcome	What it means in practice
No anonymous callers	A service will not answer a request unless the caller presented a valid, unexpired, non-revoked certificate issued by our own CA. Reaching the port is no longer enough to get a response.
No readable traffic	Anything captured on the wire is encrypted with modern authenticated ciphers, so it cannot be read and cannot be silently modified.
No impostor servers	A caller checks the name in the certificate it is shown. If DNS or routing sends it somewhere unexpected, the connection is refused rather than the data being handed over.
No expiry outages	Certificates renew themselves. Renewal happens with a 30-day safety margin, and monitoring alerts if renewal has not happened.
Fast blast-radius control	If one service is compromised, its certificate can be revoked and its identity refused, without touching any other service.
Audit evidence	Every issuance is logged: which identity, which CA, when, and by whom. This is exactly what auditors ask for under standards that require encryption in transit and strong authentication.

2. Plain-English Vocabulary

Everything in this document is built from the terms below. Read this page once and the rest of the document will read easily. Each term is expanded much further in Section 4.

Term	Plain-English meaning
TLS	The standard that encrypts a network connection and lets one side prove its identity. It is the S in HTTPS.
mTLS (mutual TLS)	Ordinary TLS, but both sides prove their identity, not just the server. The client must also present a certificate.
Certificate	A small signed file that says who you are, contains your public key, has a start and end date, and carries the CA's signature. It is public — it is meant to be shown to strangers.
Private key	The secret half of the pair. Never shared, never transmitted. Holding it is what actually proves you are the owner of the certificate.
Public key	The non-secret half. It lives inside the certificate and lets others verify signatures made with the private key.
Certificate Authority (CA)	The trusted issuer. It signs certificates. If you trust the CA, you trust every certificate it signed.
Root CA	The top of the trust tree. Self-signed, extremely long-lived, and kept offline because if it leaks, everything must be rebuilt.
Intermediate CA	The CA that actually does day-to-day signing, itself signed by the root. It is online, and it is replaceable.
Leaf certificate	The certificate issued to an actual service. It is at the bottom of the chain, hence the name.
Certificate chain	The path from a leaf certificate up through intermediates to a root you already trust. The chain is what proves the leaf is legitimate.
Keystore	Where a service keeps its own identity: its leaf certificate plus its private key. Answers the question "who am I?"
Truststore (CA bundle)	Where a service keeps the list of CAs it is willing to accept. Answers the question "whose word do I take?" It contains only public certificates and no secrets.
CSR	Certificate Signing Request. A formal request to the CA containing your public key and the identity you are asking for. The private key is never inside it.
SAN	Subject Alternative Name. The list of names a certificate is valid for, such as payments.prod.svc. This is the field that is actually checked — the older Common Name field is no longer used for this.
Hostname validation	Checking that the name you dialled matches a name in the SAN list. Without it, any valid certificate from your CA would be accepted for any service, which defeats most of the point.
EKU	Extended Key Usage. A flag on the certificate saying what it is allowed to be used for — serverAuth, clientAuth, or both.

Term	Plain-English meaning
Revocation (CRL / OCSP)	Cancelling a certificate before its expiry date, and the two ways of publishing that fact: a downloadable list (CRL) or a live question-and-answer service (OCSP).
Handshake	The short conversation at the start of every connection where the two sides agree on encryption and check each other's certificates.
Cipher suite	The specific set of algorithms chosen for the connection. Hardening means allowing only the strong ones.
Forward secrecy	A property where recording today's traffic is useless even if a private key leaks tomorrow, because the session keys were throwaway.
Secret manager	The encrypted, access-controlled store that holds private keys and certificates, with an audit log of every read and write.
Rotation	Replacing a certificate and key with a fresh pair before the old one expires, ideally without anyone noticing.
Hot reload	Loading the new certificate into a running process without restarting it, so live connections are not dropped.
SPIFFE ID	A standard way of writing a workload's identity as a URI, for example <code>spiffe://corp/ns/prod/sa/orders</code> . Useful because it names a workload rather than a machine.

3. Architecture Diagram, Explained Piece by Piece

Figure 1 appears on the following page, in landscape so that every label is readable. It shows the complete system in four horizontal layers. Read it from the top down: the top layer decides who is trusted, the second layer manufactures and delivers identities, the third layer is where two services actually talk, and the fourth layer watches the whole thing.

How to read Figure 1

Layer 1 (blue) — Trust. Where identities come from and who is allowed to issue them.

Layer 2 (amber) — Supply chain. How an identity is requested, stored, delivered to a workload and reloaded, without a human.

Layer 3 (green) — Runtime. Two services and the mutually authenticated tunnel between them.

Layer 4 (purple) — Assurance. The metrics, alerts and audit trail that prove the first three layers are healthy.

Solid arrows are the normal flow of data. Dashed arrows cross between layers — they are the moments where trust or key material is handed from one layer to the next.

End-to-End mTLS Architecture — Identity, Trust, Secure Channel and Automated Rotation

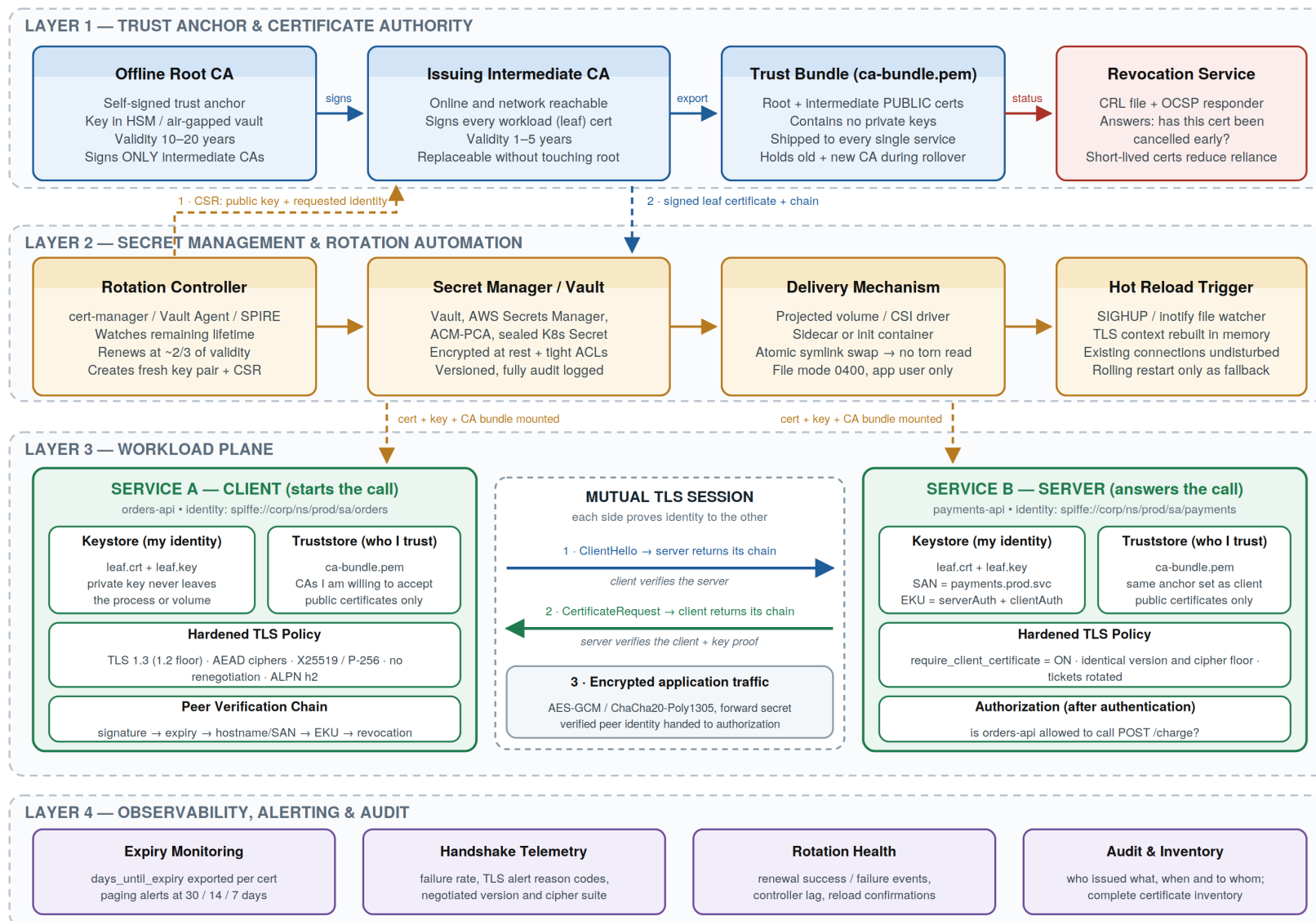


Figure 1 — The complete mTLS architecture in four layers: trust, supply chain, runtime and assurance.

3.1 Layer 1 — Trust anchor and certificate authority

This layer answers one question: whose signature counts? Nothing else in the system works until that is settled.

Offline Root CA (top-left box)

This is the single source of trust for the whole company. It signs itself, which sounds circular but simply means everyone agrees in advance to trust it. Its private key is the most valuable secret in the design, so it is kept in a hardware security module or an air-gapped machine and is powered on only for rare ceremonies. It is long-lived, typically ten to twenty years, because replacing it means touching every service in the estate. Crucially, it signs only intermediate CAs — never a service certificate directly.

Issuing Intermediate CA (second box)

This is the CA that does the real work, signing a certificate for every service, potentially thousands of times a day. Because it must be online and reachable, it is far more exposed than the root, so it is given a shorter life of one to five years. The whole reason it exists is containment: if it is ever compromised, it can be revoked and replaced while the root stays untouched, which means no service has to have its trust configuration rewritten.

Trust Bundle, `ca-bundle.pem` (third box)

This is the exported, public half of the trust decision: a plain file containing the root certificate and the intermediate certificate. It contains no private keys at all, which is why it can be copied freely to every machine. Every service reads this file to decide whose signatures it will accept. During a CA change this file temporarily holds both the old and the new CA, which is what makes a CA rollover survivable.

Revocation Service (fourth box)

Expiry handles the normal case, but sometimes a certificate must be cancelled early — a key leaked, a host was decommissioned, a service was retired. The revocation service publishes that fact, either as a downloadable list (a CRL) or as a live lookup service (OCSP). The red arrow into it shows that the intermediate CA is the thing that declares a certificate dead.

3.2 Layer 2 — Secret management and rotation automation

This layer is the factory and the delivery van. It turns a policy decision (“this workload may be called `orders-api`”) into files sitting on disk next to a running process, over and over, forever, without a human.

Rotation Controller (first box)

A background process — in practice `cert-manager`, a Vault agent, or a SPIRE agent — whose entire job is to read the expiry date on every certificate and act before it matters. When a certificate passes roughly two-thirds of its lifetime, the controller generates a brand new private key and builds a CSR. Generating a fresh key each time, rather than reusing the old one, means that even if an old key had quietly leaked, its usefulness ends at the next rotation.

Secret Manager / Vault (second box)

The certificate and its private key are stored here rather than being committed to a repository or baked into an image. The store encrypts at rest, enforces which identity may read which secret, keeps previous versions so a rollback is possible, and logs every single access. That audit log is usually the single most useful artefact when something goes wrong.

Delivery Mechanism (third box)

Getting the files from the store into the workload. In Kubernetes this is typically a projected volume or a CSI driver; elsewhere it is a sidecar or an init container. Two details matter more than they look. First, the write must be atomic — the new file is written under a temporary name and then renamed or symlinked into place — because if the application happened to read a half-written certificate it would crash or serve a broken chain. Second, permissions are tight: the key file is readable only by the application user.

Hot Reload Trigger (fourth box)

New files on disk are useless if the process is still holding the old ones in memory. The trigger — a SIGHUP signal, or a file watcher using inotify — tells the process to rebuild its TLS context from the files on disk. Connections that are already open keep running on the old material and finish normally; connections opened afterwards use the new certificate. A rolling restart is kept only as a fallback for software that genuinely cannot reload.

The two dashed arrows between Layer 1 and Layer 2

- **Arrow 1, going up:** the CSR. It carries the public key and the identity being requested. The private key deliberately never travels — it stays on the machine that generated it. This is why a compromised CA cannot leak your keys: it never had them.
- **Arrow 2, coming down:** the signed leaf certificate plus its chain, returned by the intermediate CA and stored in the secret manager.

3.3 Layer 3 — The workload plane

This is the part that actually carries business traffic. Both services are drawn with the same four internal parts, which is the point: in mTLS the client and the server are symmetric. Each one is both a prover and a verifier.

Keystore — my identity

Holds this service's own leaf certificate and its matching private key. The private key never leaves the process or the mounted volume; it is never sent over the network, not even during the handshake. What travels is a signature made with it.

Truststore — who I trust

Holds the CA bundle. It is worth being precise about the difference, because mixing these two up is the single most common configuration error in mTLS: the keystore is about proving yourself and contains a secret; the truststore is about judging others and contains no secrets at all. Both services carry the same bundle, which is what lets them recognise each other.

Hardened TLS Policy

The explicit configuration of what is allowed: TLS 1.3 preferred with 1.2 as the absolute floor, authenticated encryption ciphers only, modern elliptic curves, renegotiation disabled, and ALPN set so the protocol is agreed rather than guessed. This block appears on both sides on purpose — if only the server is hardened, a weak client can still drag the connection down to whatever they both support.

Peer Verification and Authorization

The client box lists the verification chain it runs on the server's certificate: signature, dates, hostname and SAN, EKU, revocation. The server box shows the step that comes after: authentication established who the caller is, and authorization then decides what that caller is allowed to do. A valid certificate is a passport, not a master key.

The middle panel — the mutual TLS session

Three numbered stages. The server proves itself first and the client verifies it. Then the client proves itself and the server verifies it. Only after both have passed does encrypted application traffic flow, and the verified peer identity is handed to the authorization layer. Figure 2 expands this panel into a full message-by-message sequence.

3.4 Layer 4 — Observability, alerting and audit

The three layers above are only trustworthy if you can see them working. This layer is what turns a design into something operable.

- **Expiry monitoring:** every certificate exports a days-until-expiry number. Alerts fire at thirty, fourteen and seven days. If rotation silently stops, this is what catches it — weeks before any customer is affected.
- **Handshake telemetry:** the rate of failed handshakes and the TLS alert reason for each one. A sudden rise in `unknown_ca` almost always means a trust bundle was distributed incorrectly.
- **Rotation health:** renewal successes and failures, how far behind the controller is, and confirmation that the process actually reloaded the new file.
- **Audit and inventory:** a complete list of every certificate that exists, who it was issued to, by which CA and when. Without an inventory you cannot answer the one question that matters during a CA migration: is anything still using the old CA?

The most common misunderstanding

Authentication and authorization are different things and this architecture keeps them separate. mTLS proves who the caller is. It says nothing about what the caller may do. A certificate is a passport, not a master key — the passport gets you through the door, and a separate policy decides which rooms you may enter.

4. Every Component in Detail

This section takes each component from Figure 1 and explains what it is, what it contains, why it exists, what it protects against, and what goes wrong when it is configured badly.

4.1 The Offline Root CA

What it is: a key pair and a self-signed certificate that sits at the very top of the trust tree. Self-signed means nobody vouches for it; it is trusted by prior agreement, which is why it is called a trust anchor.

- **What it holds:** an asymmetric key pair (commonly RSA-4096 or ECDSA P-384) and its own certificate with the CA flag set to true.
- **Where it lives:** in a hardware security module or on an air-gapped machine, powered on only during a formal signing ceremony with multiple people present.
- **Lifetime:** ten to twenty years. Long, because rotating it means touching the trust configuration of every service everywhere.
- **What it signs:** intermediate CAs only. Never a service certificate.
- **Why offline:** if the root key leaks, an attacker can mint a trusted certificate for any service in the company, and the only remedy is to rebuild the entire PKI. Keeping it offline means an attacker with full network access still cannot reach it.
- **Failure mode:** if this key is lost, no new intermediates can ever be issued; if it is stolen, the whole trust hierarchy is void.

4.2 The Issuing Intermediate CA

What it is: a CA whose own certificate was signed by the root, and which in turn signs leaf certificates for services.

- **Why it exists:** so the root can stay offline. This is the entire justification. It creates a replaceable component between the irreplaceable root and the thousands of leaves.
- **Lifetime:** one to five years, shorter than the root because it is exposed.
- **Constraints applied to it:** path length is set so it cannot create further CAs beneath itself, and name constraints can restrict it to only issuing for domains the organisation owns.
- **What it enforces at issue time:** before signing, it authenticates the requester and checks that this workload is entitled to the identity it is asking for. Without this check, any workload could request a certificate saying it is the payments service.
- **Failure mode:** if compromised, revoke it, issue a new intermediate from the root, push a bundle containing both old and new, reissue all leaves, then remove the old one. Disruptive but survivable — which is exactly why the layer exists.

4.3 The Certificate Signing Request (CSR)

A CSR is the application form. It contains the public key and the identity being requested, and it is self-signed with the matching private key to prove the requester actually holds it.

- **What it never contains:** the private key. This is the key property of the whole design — private keys are born on the machine that will use them and never travel anywhere.
- **What the CA does with it:** verifies the self-signature, authenticates the requester independently, checks entitlement to the requested name, and only then signs.
- **Common mistake:** treating the CSR as authoritative. The CA must never simply sign whatever name is in the CSR — the requested name has to be checked against a policy.

4.4 The Leaf Certificate, field by field

This is what a service actually presents. Every field below is checked by the other side, so it is worth knowing what each one does.

Field	What it means	Why it matters
Subject / CN	A human-readable name, kept mostly for legacy display.	No longer used for hostname matching. Do not rely on it.
Subject Alternative Name (SAN)	The list of DNS names, IPs or URIs this certificate is valid for.	This is the field hostname validation actually checks. If the name you dialled is not in this list, the connection is refused.
Public key	The public half of the key pair.	Used by the peer to verify the signature that proves possession of the private key.
Issuer	Which CA signed this certificate.	The starting point for building the chain back to a trusted root.
Serial number	A unique number assigned by the CA.	How a certificate is identified in a revocation list.
notBefore / notAfter	The validity window.	Checked on every handshake. Outside the window, the certificate is rejected. This is the field that causes expiry outages.
Basic Constraints	Whether this certificate is a CA.	On a leaf it must be false, otherwise the service could sign certificates for others.
Key Usage	The cryptographic operations allowed, such as digital signature and key encipherment.	Prevents a certificate being used for a purpose it was not issued for.
Extended Key Usage (EKU)	serverAuth, clientAuth, or both.	In mTLS a workload that both calls and is called needs both. A missing EKU is a classic cause of one-directional failures.
Authority Key Identifier	A pointer to the exact issuing CA key.	Makes chain building fast and unambiguous when a CA has several keys.
CRL / OCSP URLs	Where to check whether this certificate has been revoked.	Used by the peer during revocation checking.

Field	What it means	Why it matters
CA signature	The issuer's signature over all of the above.	Makes the certificate tamper-evident. Change one byte and the signature no longer verifies.

4.5 The Private Key

The one genuinely secret item in the system. Everything else — certificates, chains, bundles — is public information.

- **Generated where it is used:** on the workload's own host or in its own memory, so it never crosses a network.
- **Never transmitted:** not in the CSR, not during the handshake. What the handshake transmits is a signature computed with the key, which proves possession without revealing it.
- **Stored tightly:** file permissions 0400 owned by the application user, on a memory-backed volume where possible so it never touches durable disk.
- **Rotated with the certificate:** a fresh key every renewal, so any undetected leak has a strictly limited useful life.
- **Never logged, never in an image, never in Git:** these are the three places private keys are most often found during audits.

4.6 The Keystore

The service's own identity material: leaf certificate plus private key, and usually the intermediate certificate so it can present a complete chain. Formats vary — PEM files on Linux, a PKCS#12 or JKS file in the Java world — but the contents are the same.

- **Answers:** "who am I, and can I prove it?"
- **Contains a secret:** yes. This is the file that must be protected.
- **Typical mistake:** shipping the keystore inside a container image, which means every copy of that image everywhere shares one identity and one key.

4.7 The Truststore (CA Bundle)

The list of certificate authorities this service is willing to accept. Usually a single PEM file containing the root and the intermediate.

- **Answers:** "whose signature do I take as proof?"
- **Contains a secret:** no. Everything in it is public, which is why it can be distributed freely.
- **Should not contain:** the public internet CA list. An internal service should trust only the internal CA. Leaving the operating system's default bundle in place means any of hundreds of public CAs could mint a certificate your service would accept.

- **Must be identical across the estate:** if one service has a stale bundle, it will start rejecting perfectly valid peers. Distributing this file is a deployment concern, not an application concern.
- **Holds two CAs during a rollover:** this temporary widening is what makes changing the CA a non-event rather than an outage.

4.8 The Certificate Chain and Chain Building

A leaf certificate on its own proves nothing. What proves it is the chain: leaf signed by intermediate, intermediate signed by root, root already present in my truststore. The verifier walks that path and checks the signature at each hop.

- **The server sends the leaf plus intermediates, but not the root.** Sending the root would be pointless — if the peer does not already have it, receiving a copy from an untrusted stranger proves nothing.
- **Every certificate in the chain is date-checked, not just the leaf.** An expired intermediate breaks every leaf beneath it at once, which is why intermediate expiry is a much bigger incident than leaf expiry.
- **Incomplete chain is the classic failure.** The service sends only its leaf, forgetting the intermediate. It works on the developer’s laptop, because that laptop happens to have the intermediate cached, and fails in production.
- **Depth is deliberately shallow.** Root, one intermediate, leaf. Every extra layer is another thing that can expire or be misconfigured.

4.9 Hostname and SAN Validation

This is the check that makes a certificate mean something specific rather than merely being genuine. After confirming the chain is valid, the client asks a second question: is this certificate for the service I actually meant to call?

Why this single check carries so much weight

Suppose the check is skipped. Every service in the company holds a certificate signed by the same internal CA, so any one of them can now impersonate any other. A compromised low-value reporting service could present its own perfectly valid certificate and be accepted as the payments service. Chain validation alone proves the peer is “someone we issued a certificate to”. Hostname validation is what turns that into “the specific someone I intended to reach”.

- **What is compared:** the name the client dialled against the SAN list in the certificate. The Common Name field is ignored by modern libraries.
- **Wildcards:** a wildcard such as *.prod.svc matches one label only, and is best avoided internally because it dilutes identity.
- **SPIFFE alternative:** in a service mesh the identity is a URI SAN such as spiffe://corp/ns/prod/sa/orders, and the check becomes a URI match. This is stronger because it names a workload rather than a network location, so it survives rescheduling and IP changes.

- **The dangerous shortcut:** setting `InsecureSkipVerify`, `verify=false`, or an equivalent flag while debugging, and then leaving it in. This silently disables the check while everything still appears to work.

4.10 Revocation — CRL and OCSP

Expiry retires a certificate on schedule. Revocation kills one early, when a key leaks or a service is retired.

- **CRL:** a signed list of revoked serial numbers, downloaded periodically. Simple and offline-friendly, but it can grow large and it is only as fresh as the last download.
- **OCSP:** a live query asking about one specific certificate. Fresh, but it adds a network call to the handshake path and creates a dependency that can itself fail.
- **OCSP stapling:** the server fetches its own signed status periodically and attaches it to the handshake. The client gets fresh status with no extra round trip. This is the preferred approach.
- **Short lifetimes as the real answer:** with certificates valid for hours or days, a revoked identity disappears on its own almost immediately. Most modern designs treat short lifetimes as the primary control and revocation as a backstop.
- **Hard-fail versus soft-fail:** a genuine trade-off. Hard-fail refuses the connection if revocation status cannot be obtained, which is secure but turns an outage of the revocation service into an outage of everything. Soft-fail continues, which is available but can be exploited by an attacker who blocks the check.

4.11 Hardened TLS Policy

TLS is highly configurable, and its defaults are often chosen for compatibility with old software rather than for security. Hardening means removing every option that is weak, and applying the same settings to both ends.

Setting	Policy	Reason in plain terms
Protocol version	TLS 1.3 preferred, TLS 1.2 as the absolute floor. TLS 1.0 and 1.1 and all SSL versions disabled.	The old versions have known weaknesses. TLS 1.3 also removes the weak options entirely, so there is less to misconfigure.
Cipher suites	Authenticated encryption only — AES-GCM or ChaCha20-Poly1305. No RC4, no 3DES, no CBC-mode suites, no export or NULL suites.	Authenticated encryption both hides the data and detects tampering. Older modes protect confidentiality but not integrity, which has led to real, practical attacks.
Key exchange	Ephemeral Diffie-Hellman only (ECDHE), on modern curves such as X25519 or P-256.	Gives forward secrecy: session keys are thrown away, so traffic recorded today cannot be decrypted by a key stolen tomorrow.
Renegotiation	Disabled.	A historical source of vulnerabilities and denial-of-service, with no benefit in this design.
Compression	Disabled.	TLS-level compression leaks information about the plaintext.

Setting	Policy	Reason in plain terms
Session resumption	Allowed, with regularly rotated ticket keys.	Saves a round trip on repeat connections. If the ticket key is never rotated, it quietly undoes forward secrecy.
ALPN	Set explicitly, for example h2.	The application protocol is agreed inside the protected handshake rather than guessed afterwards.
Client certificates	Required, not optional.	This one setting is what turns TLS into mTLS. Set to optional, the server will happily serve anonymous callers.
Trust store	Internal CA only; the public CA list removed.	Prevents any of hundreds of public CAs from being able to mint a certificate your service would accept.
Both sides configured	The same flavor on client and server.	TLS negotiates the best both parties support. Hardening only the server means a weak client still drags the connection down.

4.12 The mTLS Enforcement Point

Somewhere, some piece of code has to actually perform the handshake. There are three common choices and the trade-off is the same each time: how much the application has to know.

Approach	How it works	Trade-off
In the application	The service configures its own TLS context using its language's library.	Maximum control and no extra moving parts, but the policy is now duplicated in every language and every repository, and hardening drifts over time.
Sidecar / service mesh	A proxy next to each workload terminates and originates mTLS; the application speaks plain HTTP to localhost.	Consistent policy everywhere, identity and rotation handled centrally, and no application changes. Costs a proxy per workload and a control plane to operate.
Gateway or load balancer	mTLS is enforced only at a boundary.	Easy to introduce, but traffic behind the boundary is unprotected, so it does not deliver service-to-service security on its own.

4.13 The Secret Manager

The system of record for certificates and private keys. It replaces the three bad habits it exists to eliminate: keys in Git, keys baked into images, and keys pasted into environment variables.

- **Encryption at rest:** the store is encrypted with a master key it manages separately, so a stolen disk yields nothing.
- **Identity-based access control:** policies say which workload identity may read which path. A service can read only its own material.
- **Versioning:** old versions are retained briefly, which makes rollback possible if a bad certificate is issued.

- **Full audit log:** every read and write recorded with who, what and when.
- **Dynamic issuance:** the strongest mode, where the store issues a short-lived certificate on demand instead of holding a long-lived one. A stolen credential then expires on its own within hours.
- **Leases and automatic expiry:** credentials that are not renewed simply die, so forgotten access does not linger.

4.14 The Rotation Controller

The component that makes expiry a non-event. It is a control loop: observe current state, compare with desired state, act on the difference, repeat.

- **What it watches:** the notAfter date of every certificate it manages, plus any request for a new identity.
- **When it acts:** at roughly two-thirds of the certificate's lifetime. For a 90-day certificate that is day 60, leaving a 30-day margin.
- **Why two-thirds and not the last day:** if renewal fails, the margin gives the automation many further attempts and gives humans weeks of alerts. Renewing at the last moment converts a small, recoverable failure into an outage.
- **Jitter:** renewal times are spread randomly so that thousands of workloads do not all renew in the same minute and overwhelm the CA.
- **Retries with backoff:** transient CA failures are retried automatically with increasing delays.
- **Idempotence:** running the loop twice does no harm, which is what makes it safe to restart the controller at any time.

4.15 The Delivery Mechanism

Moving material from the secret manager onto the workload.

- **Atomic writes:** write to a temporary file, then rename or swap a symlink. Rename is atomic on POSIX filesystems, so a reader sees either the old file or the new file, never a half-written one.
- **Memory-backed volumes:** tmpfs where possible, so keys never reach durable storage or a backup.
- **Least-privilege permissions:** 0400 for the key, owned by the application user, not world-readable.
- **Same path every time:** the application reads a fixed path and never needs to know that the file behind it changed.

4.16 The Reload Mechanism

The step that is most often forgotten, and the reason for the classic complaint “the certificate was renewed but the service is still serving the old one”.

- **Why it is needed:** most TLS libraries read certificate files once at start-up and cache the parsed result in memory. Changing the file on disk therefore changes nothing until the process is told.

- **File watching:** the process watches the directory and rebuilds its TLS context when the files change. Cleanest option.
- **Signal:** an external tool sends SIGHUP and the process reloads. Common for proxies and web servers.
- **Per-connection callback:** the TLS library asks for the current certificate at the start of each handshake, so the newest material is picked up automatically.
- **Rolling restart:** the fallback for software that cannot reload. Safe if done gradually with health checks, but it churns connections and should not be the normal path.
- **Why live connections are unaffected:** certificates are checked during the handshake, not continuously. A session established with the old certificate remains valid until it closes.

4.17 Observability and Alerting

Rotation automation is a background process, and background processes fail silently. Monitoring is what converts silent failure into a ticket.

Signal	What it tells you	Typical alert
Days until expiry, per certificate	Whether rotation is actually happening.	Warn at 30 days, escalate at 14, page at 7.
Renewal success and failure count	Whether the controller can reach and satisfy the CA.	Page on repeated failures for the same workload.
Handshake failure rate with alert reason	Trust and configuration problems, broken down by cause.	Page on a sharp rise, especially unknown_ca.
Negotiated protocol version and cipher	Whether anything is quietly negotiating down to a weak option.	Alert on any TLS 1.2 or below if 1.3 is the standard.
Reload confirmation	Whether the new material is actually in use in memory.	Alert if the served certificate does not match the file on disk.
Certificate inventory	What exists, issued by which CA. Essential during CA migration.	Alert if any workload still holds an old-CA certificate late in a migration.

4.18 The Authorization Layer

mTLS answers who. Authorization answers what. Keeping them separate is a deliberate design choice.

- **Input:** the verified peer identity extracted from the certificate — the SAN or SPIFFE ID. Because it came from a verified certificate, it cannot be forged by the caller, unlike a header.
- **Policy:** a rule such as: orders-api may call POST /charge; reporting-api may call GET /transactions only.
- **Default:** deny. Anything not explicitly permitted is refused.
- **Why not use certificates for authorization:** because encoding permissions into a certificate means reissuing certificates every time a permission changes. Identity is stable; permissions are not.

5. Workflow A — The mTLS Handshake

This is what happens on every single connection, usually in a few milliseconds. Figure 2 on the next page shows the full message sequence, and the explanation follows it.

The three things the handshake achieves

1. Both sides agree on how to encrypt, choosing from a hardened list where every option is strong.
2. Both sides prove their identity — by presenting a certificate and, crucially, by signing something with the matching private key so a stolen certificate alone is useless.
3. Both sides derive shared session keys that are thrown away when the connection ends, so recorded traffic stays unreadable even if a long-term key later leaks.

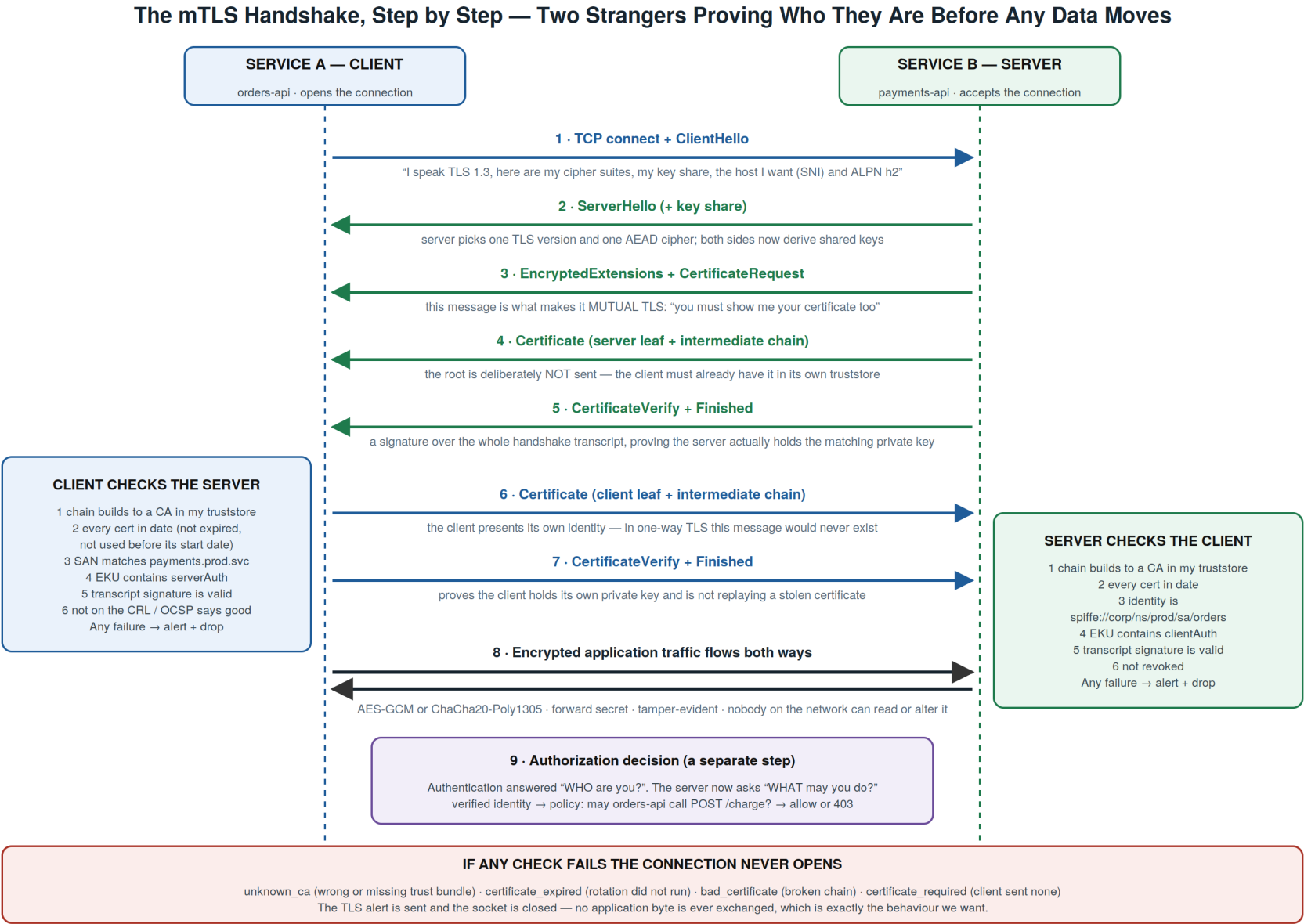


Figure 2 — The mTLS handshake: every message, every check, and what happens when a check fails.

5.1 The handshake, message by message

The numbering below matches the numbers in Figure 2.

Step 1 — TCP connection and ClientHello

The client opens an ordinary TCP connection and sends the first TLS message. It is a proposal, and it is sent in the clear because there is no shared key yet. It lists the TLS versions the client supports, the cipher suites it is willing to use, a key share for the key exchange, the hostname it is trying to reach (SNI), and the application protocol it wants (ALPN).

- **Hardening applies here:** the client offers only strong options. If the client never offers a weak cipher, the connection can never use one, regardless of what the server would have accepted.

Step 2 — ServerHello and key share

The server picks exactly one protocol version and one cipher suite from what the client offered, and returns its own key share. At this point both sides can independently compute the same shared secret without ever having sent it, and everything from here on is encrypted.

- **Forward secrecy comes from here:** the key shares are ephemeral and discarded at the end of the session. Even someone who later obtains a service’s private key cannot go back and decrypt recorded traffic.
- **Hardening applies here too:** if the server’s allowed list and the client’s offered list have no strong option in common, the handshake fails. A failed handshake is the correct outcome — far better than a successful weak one.

Step 3 — EncryptedExtensions and CertificateRequest

This is the message that makes the connection mutual. The server tells the client it must also present a certificate, and may include a list of acceptable CA names to help the client choose the right one. In ordinary one-way TLS this message simply does not exist, and that single difference is the whole of mTLS.

- **Configuration detail that matters:** many servers offer an “optional” client certificate mode. In that mode a caller presenting nothing is still served. The setting must be required, not optional, or the entire control is decorative.

Step 4 — The server sends its certificate chain

The server sends its leaf certificate and any intermediates needed to reach a trusted root. It deliberately does not send the root: the client must already have that in its own truststore, because a root handed to you by an unverified stranger proves nothing.

- **Where incomplete chains bite:** if the intermediate is omitted, clients that happen to have it cached still work while clients that do not will fail. This produces the maddening symptom of a service that works from one machine and not another.

Step 5 — CertificateVerify and Finished

The server signs a hash of the entire handshake so far using its private key. This is the step that actually proves identity. A certificate is public and can be copied by anyone; only the holder of the matching private key can produce this signature. Because the signature covers the whole transcript of this specific connection, it cannot be replayed on another one.

The client now verifies the server (left-hand panel in Figure 2)

1. **Chain** — build a path from the presented leaf up to a CA in my truststore, checking the signature at each hop.
2. **Dates** — check notBefore and notAfter on every certificate in the chain, not only the leaf.
3. **Name** — check that the hostname I dialled appears in the certificate's SAN list. This is what stops any valid certificate being accepted for any service.
4. **EKU** — check the certificate is marked for server authentication.
5. **Signature** — verify the CertificateVerify signature over the handshake transcript.
6. **Revocation** — check the certificate has not been cancelled early, via CRL, OCSP or a stapled response.

If any one of these fails, the client sends a TLS alert and closes the connection. No application data has been exchanged at this point, which is precisely the behaviour that is wanted.

Steps 6 and 7 — The client proves itself

The client now does exactly what the server just did: it sends its own leaf certificate and intermediates, then signs the handshake transcript with its own private key. The symmetry is the point. In mTLS neither party is privileged; both prove and both verify.

The server now verifies the client (right-hand panel in Figure 2)

The same six checks, with two differences. The EKU checked is clientAuth rather than serverAuth, and the identity check is usually not a hostname — a client has no hostname a server dialled — but a match of the client's SAN or SPIFFE ID against a list of identities the server is willing to talk to.

Step 8 — Encrypted application traffic

Only now does the first byte of business data move. It is encrypted with the negotiated authenticated cipher, so it cannot be read and cannot be altered without detection, and it is protected by forward secrecy.

Step 9 — Authorization

Authentication has finished; it answered who. The server now extracts the verified identity from the client's certificate and passes it to the authorization layer, which answers what. Because that identity came from a cryptographically verified certificate rather than a header, the caller cannot forge it.

5.2 What each check actually protects against

Check	Attack it prevents	What happens if you skip it
Chain validation	Someone presenting a self-made certificate.	Anyone can generate a certificate saying anything and be believed.
Date validation	Use of certificates that should have been retired.	Old, possibly leaked credentials keep working forever.
Hostname / SAN validation	One valid internal service impersonating another.	Every service in the estate can impersonate every other, because they all hold certificates from the same CA.
EKU validation	A client certificate being used to run a server, or the reverse.	Certificates get used for purposes they were never authorised for.
Signature over the transcript	Replay of a copied certificate by someone who does not hold the key.	Presenting a public certificate would be enough to impersonate its owner.
Revocation check	Continued use of a certificate after a known compromise.	A leaked key stays usable until its natural expiry.
Requiring a client certificate	Anonymous callers.	Any process that can reach the port gets served.
Cipher and version floor	Downgrade to a broken algorithm.	Traffic may be encrypted with something already known to be weak.

6. Workflow B — Automated Certificate Rotation

Certificates expire on purpose: a short life limits the damage a leaked key can do. The cost of that safety is that something must replace them, reliably, forever. Doing that by hand is how outages happen, so the entire cycle is automated. Figure 3 on the next page shows the lifetime, the loop, and the harder case of rotating the CA itself.

Why expiry causes such damaging outages

Certificate expiry is unusual among failure modes in three ways, and together they make it far worse than its simple cause suggests.

It is silent — nothing is wrong right up to the second it breaks, so there is no gradual degradation to notice.

It is simultaneous — every instance issued at the same time expires at the same time, so it is not one host failing but all of them.

It is confusing — the symptom is a connection failure, so the first hour is often spent looking at the network rather than at a date field.

Automated Certificate Rotation — How Expiry-Driven Outages Are Designed Out of the System

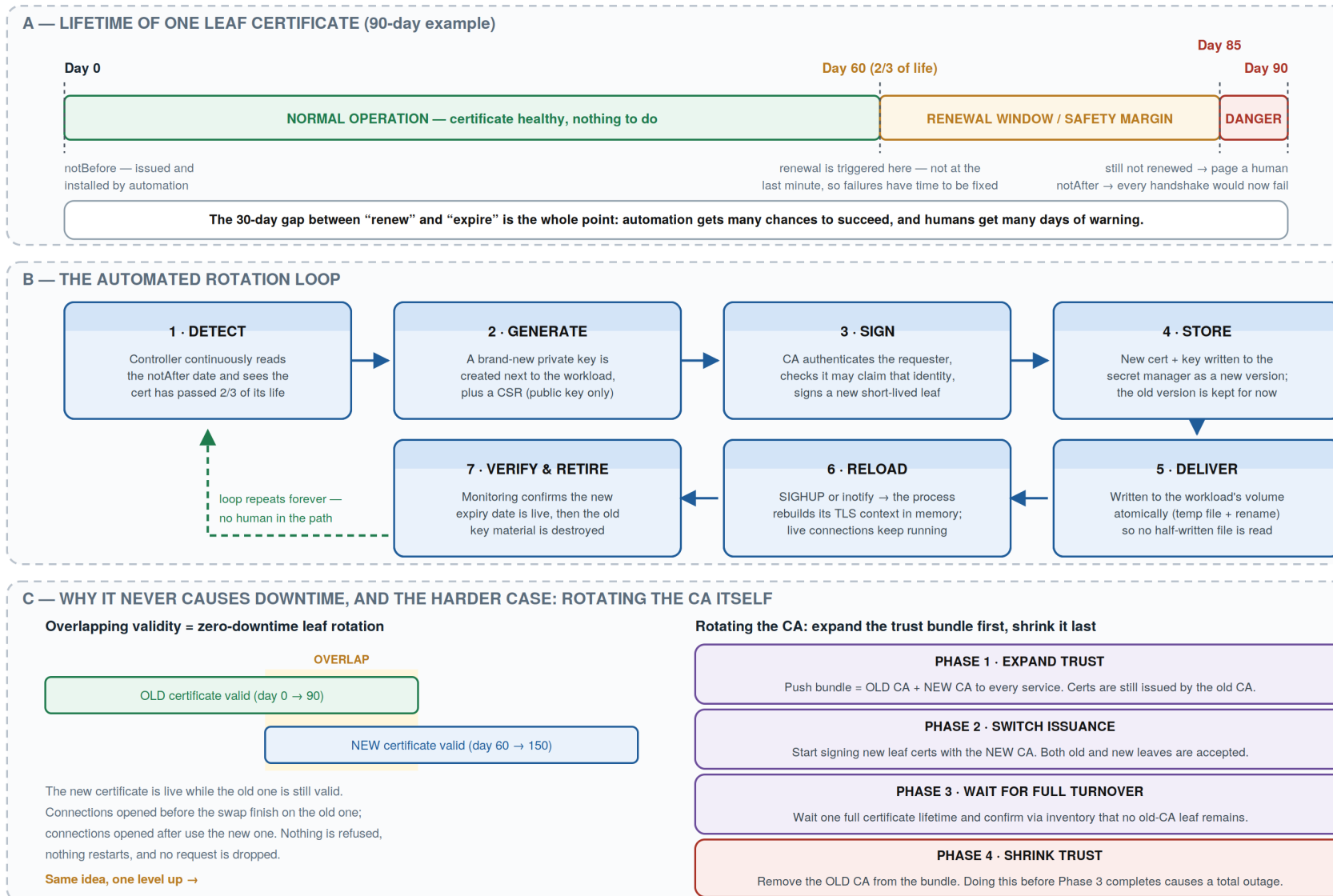


Figure 3 — Certificate lifetime, the automated rotation loop, overlapping validity, and safe CA rollover.

6.1 Panel A — The lifetime of one certificate

The bar in Figure 3 shows a 90-day certificate, a common choice. It has three zones.

- **Day 0 to day 60 — normal operation.** The certificate is healthy and nothing needs to happen. Automation simply keeps checking.
- **Day 60 to day 85 — the renewal window.** Renewal is triggered at two-thirds of the lifetime. This is the safety margin, and it is the single most important number in the design. It means a failed renewal has 30 days of retries and human attention available before anything actually breaks.
- **Day 85 to day 90 — the danger zone.** If renewal still has not succeeded, a human is paged. Past day 90 every handshake using this certificate fails.

Choosing the lifetime is a trade-off. Shorter lifetimes limit the damage from a leaked key and make revocation almost unnecessary, but they demand more from the automation and the CA. Longer lifetimes are gentler on infrastructure but leave a stolen key useful for longer. Ninety days with renewal at sixty is a common middle point; service meshes routinely go much shorter, to hours, because their automation is fully in-path and reliable.

6.2 Panel B — The seven steps of the rotation loop

1. **Detect** — The controller reads the `notAfter` date on every certificate it manages and notices that one has passed two-thirds of its life. This is a continuous loop rather than a scheduled job, so a controller that was down for a while catches up as soon as it returns.
2. **Generate** — A brand-new private key is created on the workload's own host, and a CSR is built containing the new public key and the requested identity. A fresh key every time means an undetected leak of an old key has a strictly bounded useful life. The private key does not move.
3. **Sign** — The CA authenticates the requester, checks it is entitled to the identity in the CSR, and issues a new leaf certificate with a new serial number and a fresh validity window. Entitlement checking here is what stops any workload from asking for the payments identity.
4. **Store** — The new certificate and key are written to the secret manager as a new version. The previous version is kept for a short period so a rollback is possible if the new certificate turns out to be wrong.
5. **Deliver** — The material is written into the workload's volume atomically — temporary file, then rename or symlink swap — so a reader can never observe a half-written file. Permissions are set so only the application user can read the key.
6. **Reload** — The process is signalled or notices the file change, and rebuilds its TLS context in memory. Connections already established continue on the old material and finish normally; new connections use the new certificate. Nothing restarts and no request is dropped.
7. **Verify and retire** — Monitoring confirms the new expiry date is now being served, and only then is the old private key destroyed. Verifying before destroying is what prevents the worst case, where the old material is deleted but the new material was never actually loaded.

The loop then repeats, indefinitely, with no human in the path. That is the whole objective: expiry stops being an event that anyone has to remember.

6.3 Panel C, left — Why rotation causes no downtime

The reason a swap can happen under live traffic is that the old and new certificates are valid at the same time. The new one is issued at day 60 while the old one remains valid until day 90, so for 30 days both are acceptable to any peer.

- **Connections already open** were authenticated at their start and are not re-checked mid-stream, so they simply continue.
- **Connections opened after the swap** use the new certificate, which every peer already trusts because it comes from the same CA.
- **Peers need no coordination** at all. They are validating against the CA in their truststore, not against a specific certificate, so they neither know nor care that the leaf changed.

The one thing that breaks this property

Certificate pinning — configuring a peer to accept one exact certificate rather than any certificate from the CA. Pinning turns every routine rotation into a coordinated change across every client, and it is the most common reason an otherwise well-designed rotation system causes outages. Pin to the CA if you must pin at all.

6.4 Panel C, right — Rotating the CA itself

Rotating a leaf certificate is routine. Rotating the CA is the risky operation, because the CA is the thing everyone trusts. Done in the wrong order it takes down everything at once. The rule is: expand trust first, shrink it last.

1. **Expand trust** — Distribute a bundle containing both the old and the new CA to every service. Nothing changes functionally — certificates are still issued by the old CA — but every service is now prepared to accept the new one. Wait until you have confirmed the new bundle is everywhere.
2. **Switch issuance** — Begin signing new leaf certificates with the new CA. As certificates rotate naturally, the estate drifts across. Old-CA and new-CA leaves coexist and both are accepted, because every truststore holds both anchors.
3. **Wait for full turnover** — Wait at least one full certificate lifetime, then use the inventory to confirm that not a single workload still presents a certificate from the old CA. This confirmation step is not optional.
4. **Shrink trust** — Only now remove the old CA from the bundle. Doing this before step 3 is genuinely complete causes an immediate, total, estate-wide outage, because every service still holding an old-CA certificate is instantly rejected by every peer.

The same pattern appears everywhere in distributed systems

Add the new thing, wait for everything to accept it, migrate, verify nothing uses the old thing, then and only then remove the old thing. It is the same shape as a database column migration or a protocol version upgrade. Every serious mTLS outage in the wild comes from doing the last step too early.

6.5 When rotation fails

Failure	What happens	Why it does not become an outage
CA is unreachable	Renewal attempt fails.	The controller retries with backoff. There are still weeks of validity left, so it has thousands of further attempts.
Secret manager rejects the write	New certificate is issued but not stored.	The controller treats the rotation as incomplete and retries; the old certificate is untouched and still working.
Delivery fails	New files never reach the workload.	The workload keeps using the old, still-valid certificate. Monitoring sees the expiry date not moving and alerts.
Reload does not happen	New files on disk, old certificate still in memory.	The served-certificate check compares what is on disk with what is actually being served and alerts on the mismatch.
Controller itself is down	No renewals happen at all.	Expiry monitoring is deliberately independent of the controller, so it still alerts at 30, 14 and 7 days.
Everything above fails together	Certificates approach expiry.	A human is paged with weeks of warning and can issue a certificate manually using the documented break-glass procedure.

7. The Remaining Workflows

7.1 Day zero — building the PKI

Done once, carefully, usually as a formal ceremony with more than one person present and the whole thing recorded.

1. **Create the root** — Generate the root key inside a hardware security module or on an air-gapped machine, and create the self-signed root certificate with a long validity and the CA flag set.
2. **Create the intermediate** — Generate the intermediate key on the machine that will run the issuing CA, build a CSR, carry it to the root out of band, sign it, and carry the signed certificate back.
3. **Lock the root away** — Power down the root, seal the key material, and record where it is and who can authorise its use.
4. **Publish the trust bundle** — Concatenate the root and intermediate public certificates into `ca-bundle.pem` and put it under version control, since it is public and its history is useful evidence.
5. **Stand up the supporting services** — The issuing CA endpoint, the secret manager, the revocation publication point, and the rotation controller.
6. **Write the issuance policy** — Define which workload identities exist and which one each workload is entitled to request. This policy, not the CSR, is what the CA actually enforces.

7.2 Onboarding a new service

1. **Register the identity** — Add the new workload to the issuance policy, for example `spiffe://corp/ns/prod/sa/inventory`, and state which SAN values it may claim.
2. **Grant the workload a way to prove itself** — The workload needs some initial credential to authenticate to the CA — a Kubernetes service account token, a cloud instance identity document, or a hardware attestation. This bootstrap step is the foundation of everything that follows, because it is what the CA checks before it will sign anything.
3. **Attach the delivery configuration** — Add the volume mount, sidecar or agent configuration so certificates will land at the expected path.
4. **First issuance** — The controller requests the first certificate; it is signed, stored, delivered and loaded.
5. **Configure the application** — Point it at the key, certificate and CA bundle paths, require client certificates, apply the hardened policy, and enable hostname verification.
6. **Verify before trusting** — Confirm the service can call and be called, confirm that an unauthenticated call is refused, and confirm the certificate appears in the inventory and in expiry monitoring.

7.3 A normal request at runtime

Putting the whole system together in one narrative. The orders service needs to charge a customer.

- Orders resolves payments.prod.svc and opens a TCP connection.
- The TLS handshake runs. Payments presents its certificate; orders checks the chain against its CA bundle, checks the dates, checks that payments.prod.svc is in the SAN list, checks the EKU and checks revocation.
- Payments asks orders for its certificate. Orders presents it and signs the transcript. Payments runs the same checks and confirms the identity is spiffe://corp/ns/prod/sa/orders.
- Session keys are derived. The HTTP request is sent inside the encrypted tunnel.
- Payments extracts the verified caller identity and asks its policy engine whether orders may call POST /charge. The answer is yes, so the request is processed.
- The response returns through the same encrypted tunnel. Metrics record a successful TLS 1.3 handshake.
- Meanwhile, entirely independently, the rotation controller notices that the orders certificate is at day 61 of 90 and quietly begins a renewal that nobody will ever notice.

7.4 Rolling mTLS out to an existing estate

Turning on strict mTLS everywhere at once is the reliable way to cause an outage. The safe rollout has four phases and each one is reversible.

1. **Observe** — Deploy the identity infrastructure and issue certificates to everything, but enforce nothing. Nothing changes for traffic. This proves that issuance, delivery, reload and monitoring all work at full scale before anything depends on them.
2. **Permissive mode** — Servers accept both mTLS and plain connections, and record which is which. Clients begin presenting certificates. Traffic gradually moves to mTLS with no risk, and the metrics show exactly which callers are still plaintext.
3. **Strict mode, service by service** — Once a service's dashboard shows one hundred percent mTLS for a sustained period, flip that single service to require client certificates. Start with low-risk services and keep the change trivially revertible.
4. **Strict everywhere, plus authorization** — With authentication universal, layer identity-based authorization on top and move from allow-all to default-deny.

7.5 Incident — a private key is compromised

1. **Revoke** — Add the certificate serial to the CRL and update OCSP so peers stop accepting it.
2. **Reissue** — Trigger an immediate rotation for the affected workload, generating a completely new key pair.
3. **Redeploy** — Restart or reload the workload so nothing is holding the compromised key in memory.
4. **Contain** — Use the audit log to determine what that identity was authorised to reach and what it actually did, and review those systems.
5. **Verify** — Confirm the revoked serial is genuinely being rejected by peers, rather than assuming it.

If the intermediate CA is what was compromised, the scope becomes the full CA rotation described in Section 6.4, executed under time pressure. This is precisely why the offline root exists — it makes this scenario survivable rather than terminal.

8. Failure Modes and Troubleshooting

Almost every mTLS problem in practice is one of the following. The error text is what a typical TLS library reports.

Symptom	Most likely cause	Fix
unknown_ca / unable to get local issuer certificate	The peer's certificate was signed by a CA that is not in my truststore, or the peer failed to send its intermediate.	Confirm both sides carry the same current CA bundle, and confirm the server is sending the full chain and not just its leaf.
certificate has expired	Rotation did not run, or ran but the process never reloaded.	Compare the certificate on disk with the one actually being served. If they differ, the reload is broken, not the rotation.
certificate_required / peer did not return a certificate	The client is not configured to present one, or is looking at the wrong path.	Check the client's keystore path and that client certificates are enabled in its TLS configuration.
hostname mismatch	The name dialled is not in the certificate's SAN list.	Add the correct name to the SAN at issuance. Do not disable verification to make the error go away.
handshake_failure with no other detail	No cipher suite or protocol version in common.	Compare the allowed version and cipher lists on both sides. Usually one side was hardened and the other was not.
Works from one machine, fails from another	Incomplete chain. The working machine has the intermediate cached.	Configure the server to send leaf plus intermediate.
Everything failed simultaneously across the estate	A CA change, or a trust bundle rolled out wrongly.	Roll back the bundle to the version containing both CAs. This is why the bundle is version-controlled.
Certificate renewed but old one still served	The process cached the certificate at start-up and was never told to reload.	Implement file watching or a SIGHUP handler; verify with the served-certificate metric.
Renewal storms overloading the CA	Every workload was issued at the same moment and now renews at the same moment.	Add jitter to renewal timing so requests spread out.
Intermittent failures under load	Revocation checking is hard-failing when the OCSP responder is slow.	Use OCSP stapling, and reconsider hard-fail against short certificate lifetimes.

9. Design Decisions and Why They Were Made

Decision	Alternative rejected	Reasoning
Two-tier CA: offline root plus online intermediate	A single online root.	A single root that is online is a single point of catastrophic failure; if it leaks, the entire trust hierarchy must be rebuilt and every service reconfigured. The two-tier design makes the exposed component replaceable.
Mutual authentication, not server-only	One-way TLS plus network policy.	Network position is not identity. IP allow-lists break in dynamic environments and grant access to anything that lands on the right address.
Short certificate lifetimes	Long-lived certificates.	A short life caps the damage from a leaked key and makes revocation largely unnecessary, since a revoked identity disappears on its own very quickly.
Renew at two-thirds of life	Renew close to expiry.	The margin turns a failed renewal into an alert rather than an outage, and gives automation many further attempts.
Fresh key pair on every rotation	Reuse the existing key and only reissue the certificate.	Reusing a key means an undetected leak stays useful indefinitely. Rotating the key bounds the exposure.
Secret manager as the system of record	Keys in Git, in images, or in environment variables.	Only a secret manager gives encryption at rest, per-identity access control, versioning and an audit log — all of which auditors ask for and none of which a repository provides.
Hot reload rather than restart	Rolling restart on every rotation.	Restarts churn connections and make rotation a visible event, which creates pressure to rotate less often — exactly the wrong incentive.
Hostname and SAN validation always on	Chain validation only.	Without it, any certificate from the internal CA is accepted for any service, so one compromised workload can impersonate every other.
Internal CA only in the truststore	Keeping the operating system's public CA bundle.	Trusting hundreds of public CAs for internal traffic means any of them could issue a certificate your service would accept.
Authorization kept separate from authentication	Encoding permissions into certificates.	Permissions change far more often than identities. Putting them in certificates would mean reissuing certificates for every permission change.
Expiry monitoring independent of the controller	Trusting the controller to report its own health.	A monitor that shares a failure mode with the thing it monitors is not a monitor.

10. Operational Checklist

Trust and issuance

- Root CA key is offline, in an HSM or air-gapped, with documented custody and a documented ceremony procedure.
- Root signs intermediates only; no leaf is ever signed directly by the root.
- Intermediate has a path-length constraint and, where possible, name constraints.
- The CA verifies requester entitlement against a policy before signing; it never simply signs the name supplied in the CSR.
- Every issuance is recorded in an audit log that is retained and reviewable.

Distribution and configuration

- The CA bundle is version-controlled and identical across the estate.
- The public CA list has been removed from internal truststores.
- Servers send leaf plus intermediates, never the leaf alone.
- Client certificates are required, not optional, on every server.
- Hostname or SPIFFE ID verification is enabled on every client; no insecure-skip-verify flags survive anywhere in the codebase.
- TLS 1.2 is the floor and 1.3 is preferred, with authenticated encryption ciphers only, on both client and server.
- Renegotiation and TLS-level compression are disabled; session ticket keys are rotated.

Key handling

- Private keys are generated on the host that will use them and are never transmitted.
- Key files are mode 0400, owned by the application user, on a memory-backed volume where possible.
- No private key exists in any Git repository, container image, environment variable or log.
- A fresh key pair is generated at every rotation.

Rotation

- Renewal triggers at roughly two-thirds of certificate lifetime.
- Renewal timing is jittered across workloads.
- Failed renewals are retried with exponential backoff.
- Files are delivered atomically via temporary file and rename or symlink swap.
- Processes reload without restarting; the reload path has been explicitly tested, not assumed.

- Old key material is destroyed only after the new material is confirmed to be in use.

Monitoring and response

- Days-until-expiry is exported for every certificate, including intermediates and the root.
- Alerts fire at 30, 14 and 7 days, and are routed to a team that will act on them.
- Handshake failures are tracked with their TLS alert reason.
- The certificate actually being served is compared with the one on disk.
- A complete certificate inventory exists and can answer which CA issued each certificate.
- A break-glass manual issuance procedure is documented and has been rehearsed.
- CA rotation follows expand-trust, switch-issuance, verify-turnover, shrink-trust, in that order, with no step skipped.

11. Glossary

Term	Definition
ALPN	Application-Layer Protocol Negotiation. Agreeing the application protocol, such as HTTP/2, inside the TLS handshake.
Attestation	Cryptographic evidence about what a workload is, used to authenticate it to the CA before any certificate exists.
Bootstrap identity	The initial credential a workload uses to prove itself to the CA when requesting its very first certificate.
CA bundle	A file containing the trusted CA certificates; the physical form of a truststore.
Certificate chain	The ordered path from a leaf certificate through intermediates to a trusted root.
Cipher suite	The named combination of algorithms used to protect a TLS connection.
CRL	Certificate Revocation List. A signed list of certificates cancelled before their expiry.
CSR	Certificate Signing Request. A request for a certificate, containing a public key and a requested identity.
EKU	Extended Key Usage. The declared purpose of a certificate, such as serverAuth or clientAuth.
Forward secrecy	The property that session keys are ephemeral, so recorded traffic cannot be decrypted later even if a long-term key leaks.
Handshake	The initial exchange in which the two sides authenticate and agree on keys.
Hot reload	Loading new certificate material into a running process without restarting it.
HSM	Hardware Security Module. A tamper-resistant device that stores keys and performs signing without ever exporting the key.
Intermediate CA	A CA signed by the root that performs day-to-day issuance.
Keystore	Where a service keeps its own certificate and private key.
Leaf certificate	The end-entity certificate issued to a service.
mTLS	Mutual TLS. TLS in which both parties present and verify certificates.
OCSP	Online Certificate Status Protocol. A live query for whether one certificate has been revoked.
OCSP stapling	The server attaching its own recent, signed status response to the handshake.
PEM	The common base64 text format for certificates and keys.
PKI	Public Key Infrastructure. The whole system of CAs, certificates and policies.
Private key	The secret half of a key pair; possession of it is what proves identity.
Root CA	The self-signed trust anchor at the top of the hierarchy.
Rotation	Replacing a certificate and key with a new pair before the old one expires.

Term	Definition
SAN	Subject Alternative Name. The list of names a certificate is valid for; the field hostname validation checks.
Secret manager	An encrypted, access-controlled, audited store for keys and certificates.
SPIFFE ID	A URI-form workload identity, for example <code>spiffe://corp/ns/prod/sa/orders</code> .
Trust anchor	A certificate trusted by prior agreement rather than because something vouched for it.
Truststore	The set of CAs a service is willing to accept.
TLS	Transport Layer Security. The protocol that encrypts and authenticates network connections.
X.509	The standard that defines the certificate format used throughout this document.